

Code Explanation

E2E Policy Services Data Extraction Pipeline Explanation

The provided code is the main entry point for the E2E Policy Services data extraction pipeline, which is an ETL (Extract, Transform, Load) pipeline implemented using PySpark. The pipeline extracts data from source tables, transforms the data, and loads it into a target system.

Overview of the Code

The code is structured into several functions that handle different aspects of the ETL pipeline, including logging setup, Spark session initialization, data extraction, transformation, and loading. The main function, `run\_pipeline`, orchestrates the entire process.

Step 1: Setting Up Logging

The `setup\_logging` function configures the logging for the application, setting the log level to `INFO` and specifying a format for log messages.

```
def setup_logging():
    """Configure logging for the application."""
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    )
```

Step 2: Initializing the ETL Pipeline

The `run\_pipeline` function is the main entry point for the ETL pipeline. It records the start time, sets up logging, and initializes a logger.

```
def run_pipeline():
    """
    Execute the full ETL pipeline.

    Returns:
        None
    """
    start_time = datetime.now()
    setup_logging()
    logger = logging.getLogger( name )
```

Step 3: Initializing Spark Session and Extracting Source Data

The code initializes a Spark session using the `get\_spark\_session` function and extracts source data using the `extract\_source\_data` function.

```
try:
    # Initialize Spark session
    spark = get_spark_session()
    logger.info("Spark session initialized")

    # Extract source data
```

```

logger.info("Extracting source data")
source_tables = extract_source_data(spark)
logger.info("Source data extraction complete")

```

Step 4: Transforming the Extracted Data

The extracted data is transformed using the `transform\_policy\_data` function.

```

# Transform data
logger.info("Transforming data")
transformed_data = transform_policy_data(source_tables)
logger.info(f"Transformation complete. Generated {transformed_data.count()} records")

```

Step 5: Loading the Transformed Data to the Target System

The transformed data is loaded into the target system using the `load\_to\_target` function.

```

# Load data to target
logger.info("Loading data to target")
load_to_target(spark, transformed_data)
logger.info("Data loading complete")

```

Step 6: Handling Pipeline Completion and Exceptions

The code logs the completion time and duration of the pipeline. It also handles any exceptions that occur during the pipeline execution, logging the error and re-raising the exception. Finally, it stops the Spark session.

```

end_time = datetime.now()
duration = (end_time - start_time).total_seconds()
logger.info(f"Pipeline completed successfully in {duration} seconds")

except Exception as e:
    logger.error(f"Pipeline failed: {str(e)}", exc_info=True)
    raise
finally:
    if 'spark' in locals():
        spark.stop()
        logger.info("Spark session stopped")

```

Step 7: Running the Pipeline

The `run\_pipeline` function is called when the script is executed directly.

```

if __name__ == "__main__":
    run_pipeline()

```